

Học Sâu (Deep Learning)

Chương 3: Giới thiệu TensorFlow, PyTorch, JAX và Keras

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Lịch sử phát triển các Framework Học sâu
2. Phân tích Sức mạnh của từng Framework
3. Mối quan hệ giữa Keras và các Backend
4. Bài toán cốt lõi: Bộ Phân loại Tuyến tính
5. Ứng dụng nâng cao (Tổng quan kiến trúc)
6. Tổng kết

Kỷ nguyên khởi nguyên (Trước 2009)

- Các nhà nghiên cứu phải tự viết tay logic tính toán đạo hàm (thường bằng C++).
- GPU programming gần như bất khả thi do chưa có kiến trúc hỗ trợ tốt.
- Năm 2006, NVIDIA phát hành CUDA, nhưng chủ yếu phục vụ mô phỏng vật lý chứ chưa phải Machine Learning.

Sự ra đời của các Framework đầu tiên (2009 - 2015)

- **Theano (2009):** Framework đầu tiên hỗ trợ tự động tính đạo hàm (autograd) và tính toán GPU. Là "tổ tiên" của mọi framework hiện đại.
- Cuộc thi ImageNet 2012 làm bùng nổ sự quan tâm toàn cầu tới Deep Learning, thúc đẩy các framework như Torch 7 (Lua) và Caffe (C++).
- **Keras (Đầu 2015):** Ra mắt với tư cách là thư viện Deep Learning cấp cao, dễ sử dụng, được xây dựng trên nền Theano.

Kỷ nguyên hiện đại (Cuối 2015 - Nay)

- **TensorFlow (Cuối 2015):** Do Google phát triển, hỗ trợ huấn luyện quy mô lớn (distributed computation).
- **PyTorch (Cuối 2016):** Do Meta (Facebook) phát triển để đối trọng, nổi bật với tính linh hoạt (Dynamic Computation Graph).
- **JAX (2018):** Một hệ thống gọn nhẹ từ Google kết hợp Autograd và trình biên dịch XLA, tối ưu vô cùng mạnh mẽ cho GPU/TPU.

Ba tính năng cốt lõi của một Framework

Để huấn luyện Deep Learning hiện đại, mọi framework (TF, PyTorch, JAX) đều phải giải quyết tốt 3 bài toán:

- ➊ Khả năng tự động tính đạo hàm cho các hàm phức tạp (Auto-differentiation).
- ➋ Khả năng tính toán số học tensor cực nhanh trên GPU/TPU thay vì CPU.
- ➌ Khả năng phân tán tính toán trên nhiều thiết bị (Multi-GPU/TPU) hoặc nhiều máy tính.

Tương lai của hệ sinh thái Frameworks

- Ngôn ngữ lập trình: Python thống trị hoàn toàn.
- Chúng ta đang sống trong thế giới Đa-Framework (Multi-framework): TF, PyTorch, JAX đều có chỗ đứng riêng.
- **Giá trị của Keras 3.0:** Giúp nhà phát triển chỉ cần viết mã một lần (Keras API), có thể linh hoạt chuyển đổi backend chạy trên bất kỳ framework nào (TF, Torch, JAX).

- Là một hệ sinh thái mã nguồn mở (open-source) đồ sộ của Google.
- **Hệ sinh thái Enterprise (Doanh nghiệp):** Gồm TFX (quản lý workflow), TF Serving (triển khai API), TF Lite (di động), TensorFlow.js (trình duyệt).
- Phù hợp cho việc đưa sản phẩm AI ra môi trường thực tế (Production-grade).

Tensors và Variables trong TensorFlow

Tensors là hằng số (không thể gán lại giá trị).

```
1 import tensorflow as tf
2 # Khởi tạo hằng số
3 x = tf.ones(shape=(2, 1))
4 y = tf.zeros(shape=(2, 1))
5 z = tf.constant([1, 2, 3], dtype="float32")
6
7 # Khởi tạo ngẫu nhiên
8 r1 = tf.random.normal(shape=(3, 1), mean=0., stddev=1.)
9 r2 = tf.random.uniform(shape=(3, 1), minval=0., maxval=1.)
10
```

Sử dụng tf.Variable

TensorFlow tensors là *không thể gán* (immutable). Để huấn luyện (cập nhật trọng số), ta phải dùng **Variable**.

```
1 v = tf.Variable(initial_value=tf.random.normal(shape=(3, 1)))
2
3 # Cap nhat gia tri
4 v.assign(tf.ones((3, 1)))
5 v[0, 0].assign(3.)
6
7 # Cap nhat cong/tru (nhu +=, -=)
8 v.assign_add(tf.ones((3, 1)))
9 v.assign_sub(tf.ones((3, 1)))
10
```

Toán học Tensor (Operations) trong TensorFlow

```
1 a = tf.ones((2, 2))
2 b = tf.square(a)    # Bình phương
3 c = tf.sqrt(a)      # Căn bậc 2
4 d = b + c           # Cộng từng phần tử (element-wise)
5
6 # Nhân ma trận (Dot product / Matmul)
7 e = tf.matmul(a, b)
8
9 # G ghép nối (Concatenate)
10 f = tf.concat((a, b), axis=0)
11
```

Tự động tính đạo hàm với GradientTape

```
1 input_var = tf.Variable(initial_value=3.0)
2 with tf.GradientTape() as tape:
3     result = tf.square(input_var) #  $y = x^2$ 
4
5 # Đạo hàm của  $x^2$  tại  $x=3$  là  $2*3 = 6$ 
6 gradient = tape.gradient(result, input_var)
7 print(gradient) # Tra về 6.0
8
```

Lưu ý: GradientTape mặc định chỉ theo dõi (watch) các `tf.Variable` có khả năng huấn luyện.

Đạo hàm cấp 2 với TensorFlow

Chúng ta có thể lồng các GradientTape để tính đạo hàm cấp cao (vd: vận tốc $\rightarrow$ gia tốc).

```
1 time = tf.Variable(0.0)
2 with tf.GradientTape() as outer_tape:
3     with tf.GradientTape() as inner_tape:
4         position = 4.9 * time**2
5         speed = inner_tape.gradient(position, time)
6
7 acceleration = outer_tape.gradient(speed, time) # 9.8
8
```

Biên dịch để tăng tốc (Just-In-Time Compilation)

Thực thi mặc định của Python (Eager Execution) khá chậm. Có thể dùng `@tf.function` để biên dịch hàm Python thành một Computation Graph (Đồ thị tính toán) tối ưu bằng XLA:

```
1 @tf.function(jit_compile=True)
2 def dense(inputs, W, b):
3     return tf.nn.relu(tf.matmul(inputs, W) + b)
4
```

Ưu điểm: Cực nhanh. *Nhược điểm:* Khó debug hơn vì mã chạy không còn là Python nguyên bản.

- **Thông trị mǎng Nghiên cứu (Research):** Hơn 70% các bài báo khoa học AI mới đều sử dụng PyTorch do sự thân thiện với lập trình viên (Pythonic).
- **Tính Pythonic cao:** Cú pháp rất gần gũi với NumPy.
- **Đồ thị tính toán Động (Dynamic Computation Graph):** Đồ thị được xây dựng on-the-fly, cực dễ chèn các lệnh `print` để debug.